

Chain-of-Thought Prompting vs. Fine-Tuning for Table Reasoning in Small Language Models

1. Motivation and Goals

Language models are good at writing, summarizing, and answering general knowledge questions. But when you ask one to look at a table and answer a question that needs actual reasoning across rows and columns, results get bad fast. This is not a minor edge case. Tables are the default format for most real-world data. Medical records, government data, financial reports, research results all live in tables. A model that cannot reliably work with them is limited in practice, no matter how good it is at everything else.

There are two main ways researchers have tried to fix this. Chain-of-Thought prompting, often called CoT, guides the model through a step-by-step reasoning process at the time it answers the question. No retraining required. You just change what goes into the prompt. The other approach is supervised fine-tuning, where the model is actually trained on examples of questions and correct answers until it gets better at the task. Both have shown real results in published research, but there is a catch: almost all of that research uses very large, expensive models that a regular person or student group cannot access for free.

This project focuses on the part of the problem that has not been properly studied yet. What happens when you apply both of these approaches to small models that can run on free Colab? Which one works better? When does each one break? Are the results good enough to be useful, and can they hold up on data the model was never trained on? These are practical questions that matter to anyone trying to build something real without a large compute budget.

The four main goals are: run a controlled comparison of CoT prompting and fine-tuning on the same small models, dig into where and how each method fails, test generalization to a completely different dataset, and make sure everything is reproducible on free Colab in under two hours per condition.

2. Problem Statement

Table-based question answering is harder than it looks. On the surface it seems like a reading comprehension problem. In practice it requires a sequence of steps that all have to work correctly. The model has to understand what the question is actually asking. It has to locate the right part of the table, which might involve scanning dozens of rows and columns. It has to pick the right operation, whether that is filtering, summing, comparing, or sorting, and then it has to return the answer in the correct format. A failure at any one of those steps produces a wrong answer, and the model usually has no way to signal that something went wrong. It just returns whatever it computed with full confidence.

CoT prompting addresses this by showing the model examples of how to reason through the problem step by step before asking it to solve a new one. The model reads the examples and attempts to follow the same pattern. The advantage is that no training is needed and the approach is flexible across different table formats and question types. The risk is that small models often do not have enough capacity to reliably follow complex reasoning examples. They pick up surface patterns from the demonstrations without actually understanding the logic behind them.

Fine-tuning takes a different route. The model is trained directly on pairs of questions and correct answers from a specific dataset. This tends to produce better accuracy on that dataset because the model has seen the distribution of questions and tables during training. The risk is overfitting. The model learns shortcuts specific to that dataset rather than generalizing reasoning skills, and when tested on a different dataset with a different table format or question style, performance can drop sharply.

Formally, the setup is: given a dataset $D = \{(q_i, T_i, a_i)\}$ where q_i is a natural language question, T_i is a table, and a_i is the correct answer, the goal is to find a model M such that $M(q_i, T_i)$ produces a_i as

accurately as possible, using minimal compute, and in a way that generalizes to new data. A secondary issue throughout is interpretability. CoT prompting produces a visible chain of reasoning that can be read and checked. Fine-tuning produces only a final answer with nothing showing how the model got there. In any deployment where trust matters, that distinction is important.

3. Novelty and Relation to Prior Work

The individual components of this project are not new. CoT prompting has been studied extensively. Table question answering has its own body of literature. Fine-tuning language models is standard practice. What has not been done is combining these in the specific way this project does, at this model size, under real compute constraints, with the evaluation design described here.

Wei et al. (2022) established CoT prompting as a technique and showed impressive gains on arithmetic and reasoning benchmarks. However, their experiments were almost entirely on models with over 100 billion parameters. They actually noted in the paper that smaller models see much weaker benefits from CoT. That is a critical finding that later work has largely glossed over. Wang et al. (2023) extended CoT with self-consistency, sampling multiple reasoning chains and taking the majority vote, again primarily on large models. Yao et al. (2023) introduced Tree of Thoughts, which is even more compute-heavy. None of these focus on tabular reasoning as the primary task.

On the table side, Pasupat and Liang (2015) created WikiTableQuestions and benchmarked early semantic parsing approaches. Chen et al. (2020) introduced TabFact for binary fact verification over tables. Herzig et al. (2020) built TAPAS, which fine-tunes BERT directly on table inputs but does not incorporate any step-by-step reasoning. UnifiedSKG from Xie et al. (2022) is the closest existing work to this project. It frames multiple structured grounding tasks in a single text-to-text setup, but it does not directly compare prompting versus fine-tuning as a controlled experiment, and it was not designed with strict compute limits in mind.

The contribution here is running that controlled comparison on Flan-T5-base (250M parameters) and Flan-T5-large (780M parameters), two instruction-tuned small models, on WikiTableQuestions and TabFact, under free Colab constraints, with a cross-dataset generalization test and a breakdown of error types across conditions. That specific combination of model size, task, evaluation design, and compute constraint has not appeared in published work.

4. Datasets and Acquisition

Both datasets used in this project are publicly available and load through the HuggingFace Datasets library with a single line of code. No account, API key, or manual download is needed.

WikiTableQuestions, published by Pasupat and Liang in 2015, contains 22,033 question and answer pairs drawn from 2,108 Wikipedia tables. The questions are genuinely challenging. They require filtering rows based on conditions, sorting to find maximums or minimums, comparing values across multiple columns, and sometimes chaining two of those operations together. This variety makes it a good primary benchmark because it tests a range of reasoning skills rather than one narrow pattern. It is licensed under Creative Commons BY-SA 4.0 and used here as the primary training and evaluation dataset.

TabFact, published by Chen et al. in 2020, contains 117,854 examples across 16,573 Wikipedia tables. The task is different from WikiTableQuestions. Instead of producing an answer, the model is given a statement and has to predict whether the statement is true or false based on the table. This format difference makes it useful as an out-of-distribution test. Models trained on WikiTableQuestions are evaluated on TabFact without any additional training. Because the question format, answer format, and table distribution all differ, strong performance on TabFact would be a real signal that the model learned to reason rather than memorize. It is MIT licensed too.

Tables are converted to a flat text format for model input. Column headers are listed first, followed by each row in sequence. This format stays within the token limits of the smaller models used here and is consistent with how prior work has handled table serialization for encoder-decoder models. For the prompting experiments, 6 to 8 hand-written examples with explicit reasoning steps are written by the project team and added to each prompt. For fine-tuning, rule-based templates automatically generate reasoning traces for training examples where the correct derivation is unambiguous. Official train, validation, and test splits are used for all experiments with no overlap between them.

5. Approach and Methodology

The project runs four experimental conditions plus a generalization test. They are ordered so that earlier conditions produce valid standalone results even if the project runs short on time.

The baseline condition runs both Flan-T5-base and Flan-T5-large with just the question and table as input and no examples or reasoning steps included. This shows what these models can do with zero extra effort and sets a floor for all subsequent comparisons. If the baseline is already surprisingly good on some question types, that shapes how to interpret results from the other conditions.

Condition A adds CoT prompting. Six to eight reasoning examples are prepended to each prompt. Each example shows a question, a table, a written-out reasoning process, and then the answer. No training occurs. Two chain formats are tested: one written in plain English and one using a more structured step-by-step notation. Testing both formats helps answer whether chain style affects how well small models can follow the demonstrations.

Condition B is supervised fine-tuning with answers only. Flan-T5-base is trained on WikiTableQuestions with just the final answers as targets. No reasoning steps are included in the training data. This is the standard fine-tuning approach and is expected to produce the highest accuracy on the WikiTableQuestions test set. It also sets up the comparison with Condition C.

Condition C adds reasoning traces to the fine-tuning targets. The model is trained on the same data but the target for each example is a reasoning chain followed by the answer. The chains are generated automatically using rule-based templates for training examples where the derivation is clear and unambiguous. The main question here is whether supervising on intermediate steps actually produces better final answers, or whether the model just learns to output longer text without improving accuracy.

The generalization test evaluates the fine-tuned models from Conditions B and C on TabFact with no TabFact training. This is the most informative test in the whole project because it directly measures transfer. A model that learned genuine reasoning patterns on WikiTableQuestions should be able to apply them to a different task format on different tables. A model that just memorized patterns will not.

Technical setup: AdamW optimizer, learning rate 0.0003, batch size 8 with gradient accumulation over 4 steps to reach an effective batch size of 32, trained for 3 epochs on a T4 GPU in Google Colab. All prompting experiments use greedy decoding with temperature set to 0 for consistent outputs. All random seeds are fixed and documented. Each condition is packaged as its own Colab notebook that runs end to end without any additional setup.

6. Evaluation Metrics and Plan

Evaluation has two layers. The first uses standard metrics from the literature so results can be compared to prior published work. The second layer looks at why models fail, which is often more informative than the top-line accuracy numbers.

- **Exact Match Accuracy:** The standard metric for WikiTableQuestions. The predicted answer must match the gold answer exactly after basic normalization including lowercasing and punctuation removal. This is the number that gets compared to published results.
- **Classification Accuracy:** For TabFact, the percentage of entailed or refuted predictions that are correct. Straightforward binary classification metric.

- F1 Score: Token-level F1 is reported alongside exact match for WikiTableQuestions. Some answers are lists or numerical ranges where exact match is too strict. F1 gives partial credit and provides a more complete picture.
- Error Type Breakdown: Every incorrect prediction across all conditions is labeled as one of three error types. Lookup error means the model returned a value from the wrong cell. Aggregation error means the model applied the wrong operation. Multi-hop error means the model failed on a question that required two reasoning steps. Comparing these distributions across conditions shows what each method is actually bad at, not just that it failed.
- Reasoning Chain Quality: For the CoT conditions, 100 randomly sampled chains are rated by two team members independently on a scale of 0, 1, or 2, where 0 is wrong or incoherent, 1 is partially correct, and 2 is fully correct. Cohen's kappa is calculated to measure how much the two raters agreed. This is manual work but necessary because exact match tells you nothing about whether the reasoning trace was sound.
- Compute Cost: Inference time per example and peak GPU memory are logged for each condition. CoT prompting produces much longer inputs than direct prompting, which has real latency and memory costs that accuracy numbers do not capture.

Each condition is run twice with different random seeds. Mean and standard deviation are reported for all metrics. For the main comparison between the best CoT result and the best fine-tuning result, McNemar's test is applied to the paired per-example predictions to determine whether the accuracy difference is statistically real or could be explained by random variation.

The project is considered successful if at least one condition exceeds the no-reasoning baseline by 5 exact match points on WikiTableQuestions, at least one condition achieves 60 percent or higher on TabFact without any TabFact training, and every condition runs to completion in under 2 hours on the free Colab T4. These thresholds are grounded in what small-model results look like in UnifiedSKG (Xie et al., 2022) and are achievable without being trivial.

References

- Chen, W., et al. (2020). TabFact: A Large-scale Dataset for Table-based Fact Verification. ICLR 2020.
- Herzig, J., et al. (2020). TAPAS: Weakly Supervised Table Parsing via Pre-training. ACL 2020.
- Pasupat, P., & Liang, P. (2015). Compositional Semantic Parsing on Semi-Structured Tables. ACL 2015.
- Wang, X., et al. (2023). Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR 2023.
- Wei, J., et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022.
- Xie, T., et al. (2022). UnifiedSKG: Unifying and Multi-Tasking Structured Knowledge Grounding. EMNLP 2022.
- Yao, S., et al. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. NeurIPS 2023.

AI Citation

We utilized AI-powered deep research tools to explore the topic further, refine our final idea, our methodologies, and assist with the final formatting for the proposal.

<https://claude.ai/share/cf3304ee-d16c-4d0a-bf7b-9d6899aa22a6>